

Seven Seas A-31

Security Audit

Feb 26, 2025

Version 1.0.0

Table of Contents

- [Introduction](#)
- [Overall Assessment](#)
- [Specification](#)
- [Source Code](#)
- [Issue Descriptions and Recommendations](#)
- [Security Levels Reference](#)
- [Issue Details](#)
- [Disclaimer](#)

Introduction

This document includes the results of the security audit for Seven Seas's smart contract code as found in the section titled 'Source Code'. The security audit performed by the Macro security team over multiple days starting February 21st.

The purpose of this audit is to review the source code of certain Seven Seas Solidity contracts, and provide feedback on the design, architecture, and quality of the source code with an emphasis on validating the correctness and security of the software in its entirety.

Disclaimer: While Macro's review is comprehensive and has surfaced some changes that should be made to the source code, this audit should not solely be relied upon for security, as no single audit is guaranteed to catch all possible bugs.

Overall Assessment

The following is an aggregation of issues found by the Macro Audit team:

Severity	Count	Acknowledged	Won't Do	Addressed
Medium	2	-	-	2
Low	2	-	-	2
Code Quality	3	-	-	3

Seven Seas was quick to respond to these issues.

Specification

Our understanding of the specification was based on the following sources:

- Discussions with the Seven Seas team.
- Available documentation in the repository.

Trust Assumptions:

Some of the reviewed decoders allow Boring Vaults to interact with protocols controlled by multisig wallets that can update important state variables or upgrade the contract to a new version. It is trusted that the owners of this multisig will act in the interest of users. It should be understood that a loss of Boring Vault funds is possible if these wallets act maliciously.

Source Code

The following source code was reviewed during the audit:

- **Spectra Decoder (from [PR 137](#))**

Commit Hash: `3b4600e77b8c9ef2c688e8b22399d2a31c9ef531`

Source Code	SHA256
<code>src/base/DecodersAndSanitizers/Protocols/SpectraDecoderAndSanitizer.sol</code>	<code>11145ddceb6dbc722bb19952caf6ef2011ba541afd2620a50e7702ce5cb6772b</code>

- **Resolv Decoder (from [PR 136](#))**

Commit hash: `1ac1f8f3a6fb88abac800a0099f42603a050b541`

Source Code	SHA256
<code>src/base/DecodersAndSanitizers/Protocols/ResolvDecoderAndSanitizer.sol</code>	<code>8378545b49e8291b2b2413076d204c3dad1b74cb1f30417978aad73d7f955bf7</code>

- **Uniswap V4 Decoder (from [PR 129](#))**

Commit hash: `d7211fbfaa54fcc389faa52dfdf73928391984c3`

Source Code	SHA256
<code>src/base/DecodersAndSanitizers/Protocols/UniswapV4DecoderAndSanitizer.sol</code>	<code>60bfc345ea90124e2f628cba75f608380b592c22dbbd70c156c34b9fefbd4e03</code>

- **EulerEVK Decoder (from [PR 77](#))**

Commit hash: `0d5b647d06dc43c32efb87bb586db82bea04e2d2`

Source Code	SHA256
src/base/DecodersAndSanitizers/Protocols/EulerEVKDecoderAndSanitizer.sol	ce5c0b5cfd96074f4deccbc2a722aa689f4f96665c3e2733ba6ccb14834f23a7

Note: This document contains an audit solely of the Solidity contracts listed above. Specifically, the audit pertains only to the contracts themselves, and does not pertain to any other programs or scripts, including deployment scripts.

Issue Descriptions and Recommendations

Click on an issue to jump to it, or scroll down to see them all.

- M-1** UniswapV4 decoder's `modifyLiquidities` function only sanitizes first two actions
- M-2** UniswapV4 decoder's `execute` function only sanitizes first three actions
- L-1** Spectra decoder doesn't support deposits with slippage protection
- L-2** Loose validation in UniswapV4 decoder's `modifyLiquidities` might lead to unwanted execution
- Q-1** Spectra decoder doesn't support `redeem` function
- Q-2** Resolv decoder doesn't support `redeem` function
- Q-3** UniswapV4 decoder's execute function only supports `SWAP_EXACT_IN_SINGLE` swaps

Security Level Reference

We quantify issues in three parts:

1. The high/medium/low/spec-breaking **impact** of the issue:

- How bad things can get (for a vulnerability)
- The significance of an improvement (for a code quality issue)
- The amount of gas saved (for a gas optimization)

2. The high/medium/low **likelihood** of the issue:

- How likely is the issue to occur (for a vulnerability)

3. The overall critical/high/medium/low **severity** of the issue.

This third part – the severity level – is a summary of how much consideration the client should give to fixing the issue. We assign severity according to the table of guidelines below:

Severity	Description
(C-x) Critical	We recommend the client must fix the issue, no matter what, because not fixing would mean significant funds/assets WILL be lost.
(H-x) High	We recommend the client must address the issue, no matter what, because not fixing would be very bad, or some funds/assets will be lost, or the code's behavior is against the provided spec.
(M-x) Medium	We recommend the client to seriously consider fixing the issue, as the implications of not fixing the issue are severe enough to impact the project significantly, albeit not in an existential manner.
(L-x) Low	The risk is small, unlikely, or may not be relevant to the project in a meaningful way. Whether or not the project wants to develop a fix is up to the goals and needs of the project.
(Q-x) Code Quality	The issue identified does not pose any obvious risk, but fixing could improve overall code quality, on-chain composability, developer ergonomics, or even certain aspects of protocol design.
(I-x) Informational	Warnings and things to keep in mind when operating the protocol. No immediate action required.
(G-x) Gas Optimizations	The presented optimization suggestion would save an amount of gas significant enough, in our opinion, to be worth the development cost of implementing it.

Issue Details

M-1

UniswapV4 decoder's `modifyLiquidities` function only sanitizes first two actions

TOPIC	STATUS	IMPACT	LIKELIHOOD
Sanitization	Fixed ↗	Medium	Medium
	Fixed ↗		

In `UniswapV4DecoderAndSanitizer`, the `modifyLiquidities` function allows for adding, removing, and modifying liquidity on UniswapV4 pools. It allows for batching multiple actions that are encoded in the first argument:

```
function modifyLiquidities(bytes calldata unlockData, uint256 /*deadline*/) ext
    // First decode the outer tuple (actions, params)
    (bytes memory actions, bytes[] memory params) = abi.decode(unlockData,
```

Reference: [UniswapV4DecoderAndSanitizer.sol#L56](#)

However, only `actions[0]` and `actions[1]` are sanitized, and the function lacks a check to prevent additional actions from being passed in the `unlockData` argument. As a result, any additional actions encoded in `unlockData` can be executed without any sanitization.

Remediation to Consider

It is recommended to restrict the execution only to supported action sequence. Thus, consider adding a check to only allow the required number of actions passed to the function.

M-2

UniswapV4 decoder's `execute` function only sanitizes first three actions

TOPIC	STATUS	IMPACT	LIKELIHOOD
Sanitization	Fixed ↗	Medium	Medium

Similar to M-1, the `UniswapV4DecoderAndSanitizer's execute` function only sanitizes the first three actions. Any additional actions being passed in `inputs[0]` remain unsanitized.

Remediation to Consider

Restrict the execution only to supported action sequence. Thus, consider adding a check to only allow the required number of actions passed to the function.

L-1

Spectra decoder doesn't support deposits with slippage protection

TOPIC	STATUS	IMPACT	LIKELIHOOD
Missing function	Fixed ↗	Low	Low

Currently, the spectra decoder implements only the basic `deposit` function without slippage protection:

```
function deposit(uint256 /*amount*/, address receiver)
```

Remediation to Consider

Add the `deposit` function with slippage protection to avoid receiving less tokens than expected.

Q-2

Loose validation in UniswapV4 decoder's `modifyLiquidities` might lead to unwanted execution

TOPIC

Sanitization

STATUS

Fixed [↗](#)

Fixed [↗](#)

Fixed [↗](#)

IMPACT

Low

LIKELIHOOD

Low

In `UniswapV4DecoderAndSanitizer`, `modifyLiquidities` misses certain checks that would ensure only supported sequences of actions are allowed. Specifically, it's recommended to add verification for:

1. for the `MINT_POSITION` operation, check that `actions[1]` is `SETTLE_PAIR` before decoding `params[1]`
2. for the `INCREASE_LIQUIDITY`, `DECREASE_LIQUIDITY`, and `BURN_POSITION` operations, the `tokenId` is encoded in `params[0]` to identify the specific pool for execution. However, this is not sanitized. The `PositionManager`'s `getPoolAndPositionInfo` could be called to receive the `PoolKey` and sanitize it's addresses.
3. for the `INCREASE_LIQUIDITY` / `DECREASE_LIQUIDITY` operation when using sub-actions `CLOSE_CURRENCY` or `CLEAR_OR_TAKE`, check for `actions[2] == CLOSE_CURRENCY` and `actions[2] == CLEAR_OR_TAKE`
4. for the `BURN_POSITION` operation, check that `actions[2] == TAKE_PAIR`

Q-4

Spectra decoder doesn't support `redeem` function

TOPIC

STATUS

QUALITY IMPACT

Missing functionality

Fixed  Medium

The spectra decoder currently implements `redeemIBT` to redeem interest-bearing tokens (IBT), but lacks support for the `redeem` function that would allow redeeming the underlying token.

Remediation to Consider

Add the [redeem function](#) with slippage protection to the decoder.

Q-2

Resolv decoder doesn't support `redeem` function

TOPIC

Missing functionality

STATUS

Fixed 

QUALITY IMPACT

Medium

The Resolv decoder implements `requestBurn`, which burns `USR` and returns `USDC` once `completeBurn` is called by a trusted entity. The Resolv protocol also offers a redeem function that burns USR tokens and returns USDC immediately in a single transaction.

Remediation to Consider

Implement the `redeem` function to support burning USR and receiving back USDC in a single transaction.

Q-3

UniswapV4 decoder's execute function only supports `SWAP_EXACT_IN_SINGLE` swaps

TOPIC

STATUS

QUALITY IMPACT

Missing functionality

Fixed [↗](#) Medium

In `UniswapV4DecoderAndSanitizer`, the `execute` function enables swapping via the UniversalRouter contract. Currently, the function only supports "Exact Input Swaps." To increase flexibility, implementing additional swap types—such as "Exact Output Swaps"—would be beneficial.

Remediation to Consider

Implement support for additional swap actions.

Disclaimer

Macro makes no warranties, either express, implied, statutory, or otherwise, with respect to the services or deliverables provided in this report, and Macro specifically disclaims all implied warranties of merchantability, fitness for a particular purpose, noninfringement and those arising from a course of dealing, usage or trade with respect thereto, and all such warranties are hereby excluded to the fullest extent permitted by law.

Macro will not be liable for any lost profits, business, contracts, revenue, goodwill, production, anticipated savings, loss of data, or costs of procurement of substitute goods or services or for any claim or demand by any other party. In no event will Macro be liable for consequential, incidental, special, indirect, or exemplary damages arising out of this agreement or any work statement, however caused and (to the fullest extent permitted by law) under any theory of liability (including negligence), even if Macro has been advised of the possibility of such damages.

The scope of this report and review is limited to a review of only the code presented by the Seven Seas team and only the source code Macro notes as being within the scope of Macro's review within this report. This report does not include an audit of the deployment scripts used to deploy the Solidity contracts in the repository corresponding to this audit. Specifically, for the avoidance of doubt, this report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. In this report you may through hypertext or other computer links, gain access to websites operated by persons other than Macro. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such websites' owners. You agree that Macro is not responsible for the content or operation of such websites, and that Macro shall have no liability to your or any other person or entity for the use of third party websites. Macro assumes no responsibility for the use of third party software and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.